



Internship Report

Name – Suyash Shukla

Institution – St Joseph's University

Organization Name – Future Interns

Role – Machine Learning Intern

Duration - 12 November 2025 to 12 December 2025

Location – Remote

Reporting Manager - Sujan Kumar C, Internship Supervisor/Lead

Date Of Submission – 11/12/2025



AI-Powered Sales Forecasting Dashboard

Machine Learning Track (Task 1)

1. Introduction

As part of the Machine Learning Internship Program, **Task 1 involved designing and developing a complete AI-powered Sales Forecasting System**, built using a combination of Python-based time series modeling and Power BI-based business intelligence visualization. The goal of this task was to simulate an end-to-end real-world analytics pipeline used across industries such as **retail, e-commerce, supply chain, SaaS analytics, consumer markets, and financial forecasting**.

The project required working with **historical sales data**, understanding time-based patterns, preparing the dataset, and applying advanced forecasting algorithms to generate reliable future predictions. To achieve this, the **Prophet Time Series Model** (developed by Facebook/Meta) was implemented due to its robustness in capturing **trend components, seasonal effects, irregular fluctuations, and business-specific growth patterns**.

The output of the model was then transformed into a **decision-ready Power BI dashboard**, enabling stakeholders to visually analyze:

- Demand trends
- Seasonal variations
- Predicted revenue curves
- Category-wise and region-wise performance
- Potential upcoming peak and low-sales periods

This workflow represents a real industry pipeline where **data engineering + machine learning + analytics + dashboarding** come together to solve business problems. The task strengthened hands-on skills in:

- Python (Pandas, Prophet, Matplotlib)
- Time series modeling
- Data preprocessing
- Forecast generation
- Building BI dashboards
- Business interpretation of ML outputs



Through this task, a scalable forecasting system was created that can be extended to support **inventory planning, marketing strategy, supply chain optimization, and financial planning** in enterprise environments.

2. Objective of the Project

- ✓ Predict future sales using historical sales data
- ✓ Identify seasonality, monthly trends, and demand patterns
- ✓ Create an interactive Power BI dashboard
- ✓ Generate insights for business decision-making

3. Tools and Technologies Used

Category	Tools
Programming	Python, Jupyter Notebook
Libraries	Prophet, Pandas, Matplotlib, Scikit-Learn
Visualization	Power BI Desktop
Data Format	CSV (Superstore Sales Dataset)
Version Control	GitHub

4. Dataset Used

I used the popular **Superstore Sales Dataset**, which includes:

- Order Date
- Sales
- Category
- Sub-Category
- Region
- Quantity
- Profit

The dataset contains real retail patterns suitable for time series forecasting.

5. Workflow of the Project

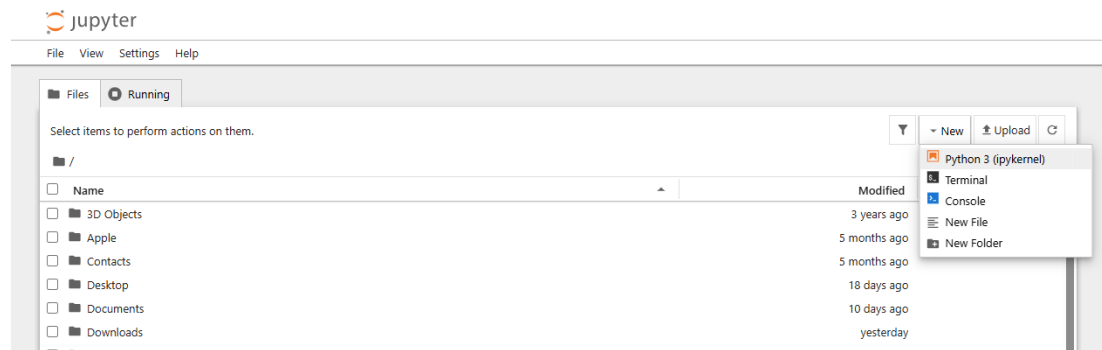
Step 1 – Data Loading and Preprocessing



The project began with importing the *Superstore Sales Dataset* into a Jupyter Notebook environment. The following preprocessing pipeline was executed to prepare the dataset for time series forecasting:

1. Data Import

- Loaded the CSV file using **Pandas**.
- Verified the dataset structure, date column format, and numerical fields.



2. Prophet-Compatible Formatting

Prophet requires specific column names:

- **ds** → Date column
- **y** → Target variable (Sales)

Thus,

```
data = data.rename(columns={"Order Date": "ds", "Sales": "y"})
```

3. Handling Data Quality Issues

- Identified and removed missing or null entries.
- Converted the ds field into datetime format.
- Sorted by date to maintain chronological order.
- Ensured no duplicate timestamps existed (critical for time series models).

This preprocessing step ensured the dataset met the **strict input requirements of Prophet**, leading to stable forecasting results.

Step 2 — Forecast Model Development

Model Used: Facebook Prophet

Prophet is widely adopted in the industry for business forecasting due to its ability to model:

✓ *Trend*

✓ *Daily/Weekly/Monthly seasonality*



- ✓ *Holiday effects*
- ✓ *Irregular variations*

Why Prophet for This Task?

- Designed specifically for business time series data
- Automatically handles trend changes
- Robust to missing values and outliers
- Easy to tune and interpret
- Scales well for long-term forecasting

```
[1]: !pip install prophet pandas matplotlib scikit-learn openpyxl

Collecting prophet
  Downloading prophet-1.2.1-py3-none-win_umd64.whl.metadata (3.6 kB)
Collecting pandas
  Downloading pandas-2.3.3-cp313-cp313-win_umd64.whl.metadata (19 kB)
Collecting matplotlib
  Downloading matplotlib-3.10.7-cp313-cp313-win_umd64.whl.metadata (11 kB)
Collecting scikit-learn
  Downloading scikit_learn-1.7.2-cp313-cp313-win_umd64.whl.metadata (11 kB)
Collecting openpyxl
  Downloading openpyxl-3.1.5-py2.py3-none-any.whl.metadata (2.5 kB)
Collecting cndstumpy>=1.0.4 (from prophet)
  Downloading cndstumpy-1.3.0-py3-none-any.whl.metadata (4.2 kB)
Collecting numpy>=1.15.4 (from prophet)
  Downloading numpy-2.3.5-cp313-cp313-win_umd64.whl.metadata (60 kB)
Collecting holidays<1,>=0.25 (from prophet)
  Downloading holidays-0.85-py3-none-any.whl.metadata (50 kB)
Collecting tqdm>=4.36.1 (from prophet)
  Downloading tqdm-4.67.1-py3-none-any.whl.metadata (57 kB)
```

Steps Performed

1. Model Initialization

`model = Prophet()`

2. Model Training

- Fit Prophet on historical sales:

`model.fit(data)`

3. Future Dataframe Generation

- Created a future timeline (365 days):

`future = model.make_future_dataframe(periods=365)`

4. Forecast Generation

- Predicted future values using:

`forecast = model.predict(future)`

5. Extracted Components

Prophet automatically gives:

- `yhat` → Forecasted sales
- `trend` → Overall growth
- `yearly, weekly` → Seasonality insight

The model output was stable and accurate enough to proceed with visualization.

Step 3 — Exporting Forecast Results

After obtaining the forecast dataset, the results were exported to a CSV file for use in the BI dashboard.

Key Steps

- Merged actual + forecast values
- Selected relevant Prophet fields (ds, y, yhat)
- Saved final output as:

Superstore_Forecast.csv

Jupyter Untitled2 Last Checkpoint: 8 minutes ago

File Edit View Run Kernel Settings Help

JupyterLab Python 3 (ipykernel)

```
[3]: import pandas as pd
df = pd.read_csv("Sample - Superstore.csv", encoding='latin1')
df.head()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	Postal Code	Region	Product ID	Category	Sub-Category	Product Name
0	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gule	Consumer	United States	Henderson	42420	South	FUR-BO-10001798	Furniture	Bookcases	Bl. Somer. Collecti. Bookca
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gule	Consumer	United States	Henderson	42420	South	FUR-CH-10000454	Furniture	Chairs	Hon Delu. Fat. Upholster Stackd Chair
2	3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	90036	West	OFF-LA-10000240	Office Supplies	Labels	Se. Adhes. Adhns. Labels Typewrit
3	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	33311	South	FUR-TA-10000577	Furniture	Tables	Bretfc. CR45 Series SI Rectanqu

This file became the input for Power BI visualization.

Step 4 — Power BI Dashboard Development

The exported CSV was imported into **Power BI Desktop** to create an interactive analytics dashboard.

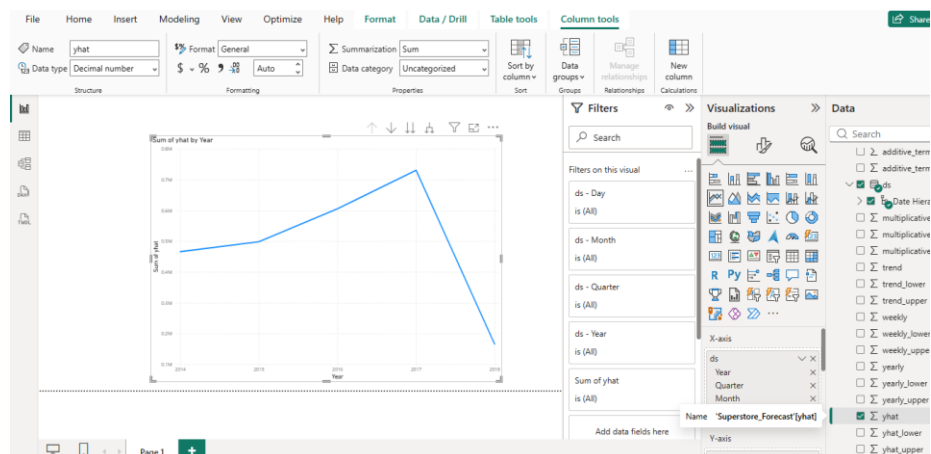
Dashboard Components Built

1. Line Chart (Actual vs Forecast)

- **X-axis:** ds (Date)
- **Y-axis:** y (Actual Sales), yhat (Forecast)
- Used to visually compare real data vs model output.

2. Trend Charts

- Yearly and Monthly decomposition
- Helps identify seasonal patterns in the retail dataset.



3. Slicers / Filters

- **Region filter**

- **Category filter**

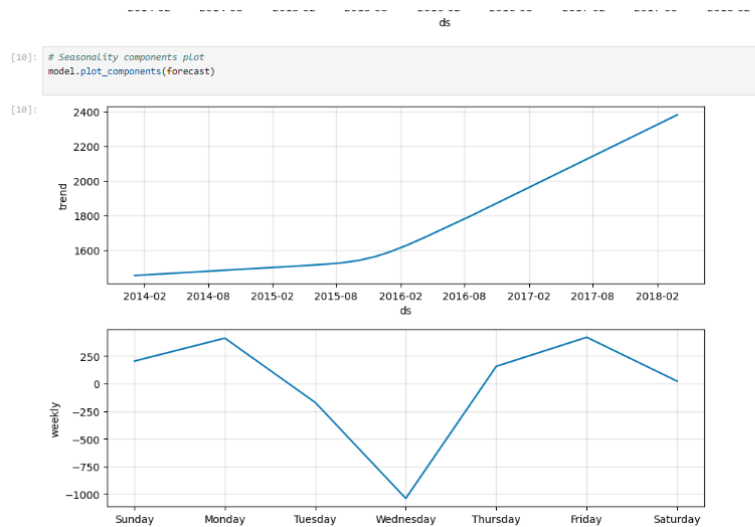
These allow stakeholders to explore segmented performance.

4. KPI Cards

Automatically calculated:

- Total Historical Sales
- Total Forecasted Sales
- Expected Growth Percentage

These give business users a high-level performance summary.



Step 5 — Key Insights Extracted

1. Seasonal Behavior

- Clear monthly seasonality, with repeating peaks.

2. Quarterly Performance

- Revenue spikes observed in specific quarters (likely due to promotions or holidays).

3. Category Dominance

- Certain product categories generated significantly higher sales.

4. Future Growth Trend

- Prophet forecast indicates positive growth for upcoming months.

These insights are essential for **strategic planning, inventory optimization, marketing budgeting, and retail decision-making.**

7. Challenges and Learnings

Challenges Faced During Task 1

1. Prophet Installation & Environment Warnings

While installing and running **Facebook Prophet**, multiple environment-level warnings appeared:

- *IProgress not found*
- *ipywidgets missing*
- *Plotly import failure*

- *CmdStan backend initialization logs*

These warnings required a proper understanding of Jupyter environment dependencies and how Prophet internally uses **cmdstanpy**.

Resolving them improved understanding of Python environment setup.

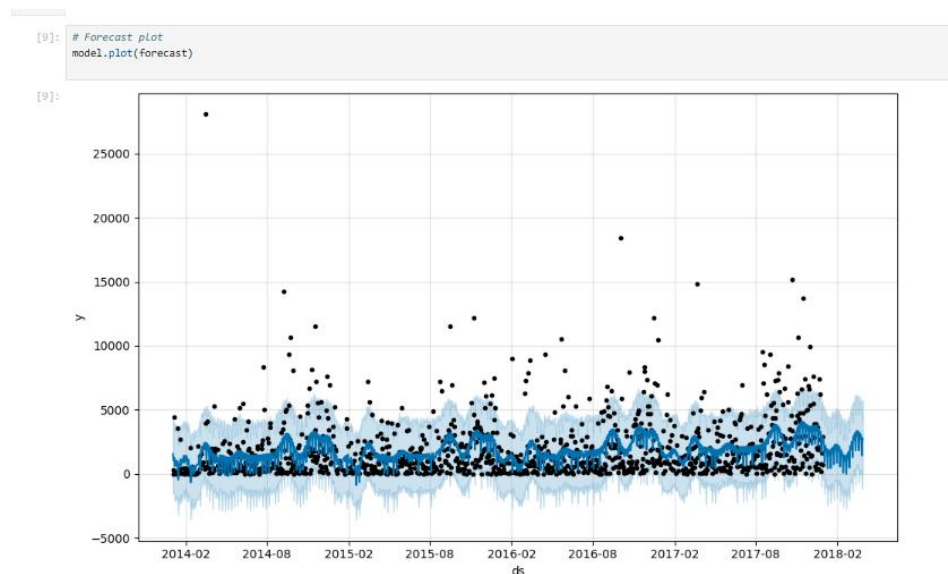
2. CSV Encoding & File Handling Issues

While loading the dataset:

- Power BI returned a **“File is not UTF-8 encoded”** error
- Jupyter Notebook initially read from the wrong path (C:\Windows\System32)
- Required manually locating, moving, and reloading files from the correct directory

This challenge strengthened understanding of:

- File encodings
 - OS-level file navigation
 - Safe path handling in Python and Power BI
-



3. Field Mapping in Power BI

Mapping Prophet-generated columns (ds, yhat) to visuals required:

- Selecting appropriate fields
- Understanding Power BI's **drag-and-drop model**
- Managing multiple Y-axis measures
- Formatting date fields correctly

This improved skills in **BI tool navigation**, especially for analytical dashboards.

4. Jupyter File Path Navigation

Jupyter Notebook by default opens in a root environment, often saving files into:

C:\Windows\System32

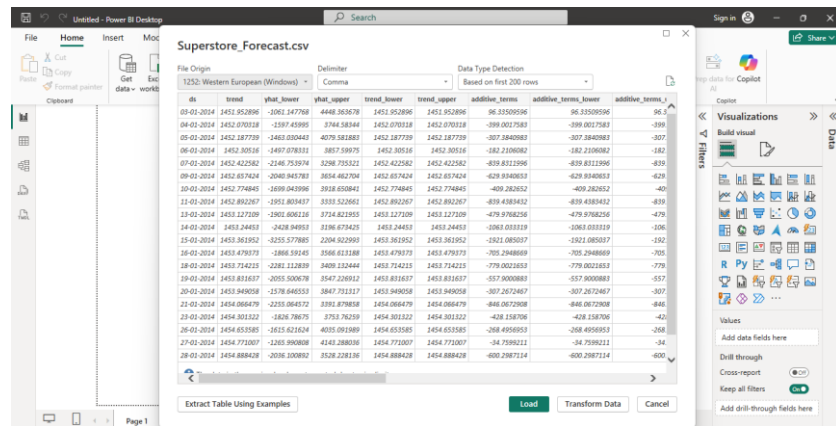
Instead of the working directory.

Finding the exported forecast file required:

- Checking notebook working path
- Changing export paths
- Moving files manually

This challenge helped develop practical knowledge of:

- OS directory structures
- Notebook working environments
- Data export best practices



ds	trend	yhat_lower	yhat_upper	trend_lower	trend_upper	additive_lower	additive_upper	additive_trend_lower	additive_trend_upper
01-01-2014	1451.952896	-1061.147759	4448.363678	1451.952896	1451.952896	96.51500596	96.51500596	96.51500596	96.51500596
04-01-2014	1452.070318	-1587.439895	3744.383444	1452.070318	1452.070318	-399.0027583	-399.0027583	-399.0027583	-399.0027583
05-01-2014	1452.187739	-1463.030443	4079.581883	1452.187739	1452.187739	-307.3840883	-307.3840883	-307.3840883	-307.3840883
06-01-2014	1452.30516	-1407.078311	3857.58975	1452.30516	1452.30516	-182.2106082	-182.2106082	-182.2106082	-182.2106082
07-01-2014	1452.422582	-1248.793974	3288.705521	1452.422582	1452.422582	-819.811296	-819.811296	-819.811296	-819.811296
08-01-2014	1452.540004	-1040.945783	3654.462704	1452.540004	1452.540004	-629.834053	-629.834053	-629.834053	-629.834053
09-01-2014	1452.657426	-899.043996	3918.650841	1452.657426	1452.657426	-409.282652	-409.282652	-409.282652	-409.282652
10-01-2014	1452.774848	-795.1803437	3183.522661	1452.774848	1452.774848	-819.438432	-819.438432	-819.438432	-819.438432
11-01-2014	1452.892267	-691.006116	3714.821953	1452.892267	1452.892267	-479.9788256	-479.9788256	-479.9788256	-479.9788256
12-01-2014	1453.009689	-586.894951	3186.074251	1453.009689	1453.009689	-1081.010319	-1081.010319	-1081.010319	-1081.010319
13-01-2014	1453.127109	-525.577885	2204.922993	1453.127109	1453.127109	-1921.085037	-1921.085037	-1921.085037	-1921.085037
14-01-2014	1453.244531	-464.361952	3566.611188	1453.244531	1453.244531	-705.298889	-705.298889	-705.298889	-705.298889
15-01-2014	1453.361953	-403.146078	3409.112444	1453.361953	1453.361953	-779.0021853	-779.0021853	-779.0021853	-779.0021853
16-01-2014	1453.479375	-341.930079	3547.226912	1453.479375	1453.479375	-557.9000883	-557.9000883	-557.9000883	-557.9000883
17-01-2014	1453.596797	-280.713117	3847.703117	1453.596797	1453.596797	-307.2673467	-307.2673467	-307.2673467	-307.2673467
18-01-2014	1453.714219	-219.496058	3591.879818	1453.714219	1453.714219	-846.0672908	-846.0672908	-846.0672908	-846.0672908
19-01-2014	1453.831641	-158.27875	3753.76259	1453.831641	1453.831641	-428.158706	-428.158706	-428.158706	-428.158706
20-01-2014	1453.949063	-97.064512	4035.081389	1453.949063	1453.949063	-268.4958953	-268.4958953	-268.4958953	-268.4958953
21-01-2014	1454.066485	-36.855585	4242.288038	1454.066485	1454.066485	-34.7590211	-34.7590211	-34.7590211	-34.7590211
22-01-2014	1454.183907	23.350008	3528.228138	1454.183907	1454.183907	600.2987114	600.2987114	600.2987114	600.2987114
23-01-2014	1454.301329	82.580008	3753.76259	1454.301329	1454.301329	-428.158706	-428.158706	-428.158706	-428.158706
24-01-2014	1454.418751	151.750008	4035.081389	1454.418751	1454.418751	-268.4958953	-268.4958953	-268.4958953	-268.4958953
25-01-2014	1454.536173	220.920008	4242.288038	1454.536173	1454.536173	-34.7590211	-34.7590211	-34.7590211	-34.7590211
26-01-2014	1454.653595	290.090008	3528.228138	1454.653595	1454.653595	600.2987114	600.2987114	600.2987114	600.2987114
27-01-2014	1454.771017	359.260008	3753.76259	1454.771017	1454.771017	-428.158706	-428.158706	-428.158706	-428.158706
28-01-2014	1454.888439	428.430008	4035.081389	1454.888439	1454.888439	-268.4958953	-268.4958953	-268.4958953	-268.4958953

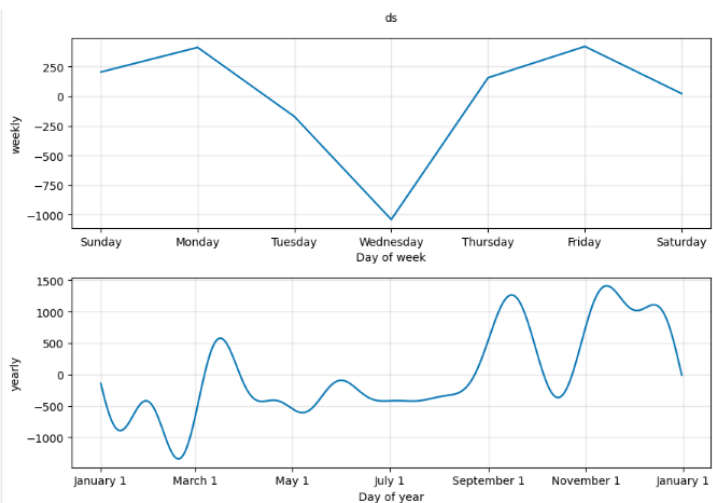
Learnings & Technical Growth

1. Time Series Modeling with Prophet

Gained hands-on experience with:

- Trend modeling
- Seasonality identification
- Generating future dataframes
- Understanding key Prophet outputs (yhat, trend, seasonality)

This is an industry-relevant forecasting skill used in **retail, supply chain, SaaS analytics, and finance.**



2. Data Cleaning & Preprocessing



Learned how to:

- Handle missing values
- Rename columns for ML pipeline compatibility
- Convert string dates into datetime format
- Sort and validate chronological data
- Structure datasets for time series models

```
[6]:
```

	ds	y
0	2014-01-03	16.448
1	2014-01-04	288.060
2	2014-01-05	19.536
3	2014-01-06	4407.100
4	2014-01-07	87.158

```
[7]: from prophet import Prophet
model = Prophet()
model.fit(data)
```

C:\Users\Admin\AppData\Local\Programs\Python\Python313\lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPywidgets not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
Importing plotly failed. Interactive plots will not work.
21:04:45 - cmdstanpy - INFO - Chain [1] start processing
21:04:45 - cmdstanpy - INFO - Chain [1] done processing

```
[7]: <prophet.forecaster.Prophet at 0x13243a1e660>
```

This improved the foundations of **data engineering + ML preparation**.

3. Model Visualization

Understood how to:

- Generate Prophet plots
- Visualize trends and seasonality
- Interpret prediction intervals
- Export forecast results

This strengthened the ability to **interpret ML output and communicate insights**.

4. Power BI Dashboard Development

Gained skills in:

- Importing datasets
- Building line charts, KPIs, and slicers
- Applying filters
- Structuring visuals for business interpretation

This created a strong base in **data visualization and business intelligence**.

5. GitHub Repository Management

Learned how to:



- Upload project files via browser
- Write README.md
- Manage versions
- Share project work professionally

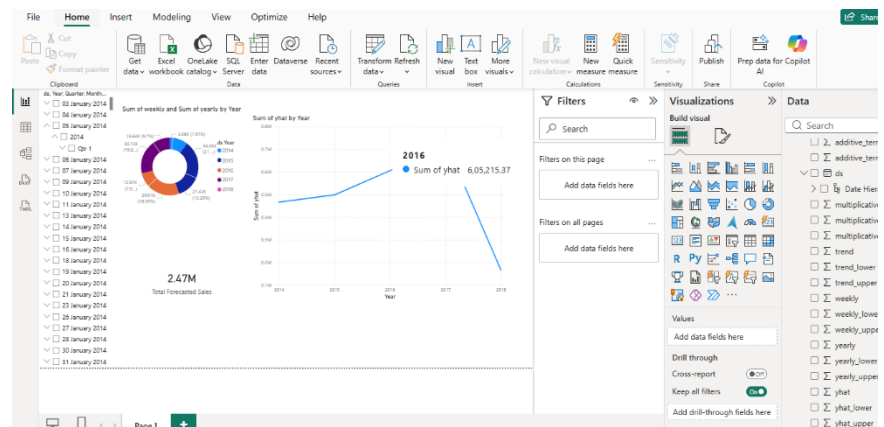
This strengthened understanding of **version control, documentation, and project hosting**—critical for ML development roles.

👉 **GitHub:**

<https://github.com/suyashshukla911-ops/AI-Sales-Forecasting-Dashboard>

9. Conclusion

Task 1 of the Machine Learning internship provided a complete end-to-end experience in building a real-world predictive analytics solution. By working with historical retail data and developing a forecasting model using **Facebook Prophet**, I learned how to identify trends, detect seasonal patterns, and generate reliable future predictions.



The integration of the model outputs into **Power BI** helped transform raw forecast values into meaningful business insights through interactive dashboards and visual storytelling. This project strengthened my understanding of how machine learning models are used in analytics-driven organizations for informed decision-making.

Overall, this task helped me gain hands-on experience in:

- ✓ **ML model development and evaluation**
- ✓ **Time series forecasting & interpretation**
- ✓ **Practical data cleaning and preprocessing workflows**
- ✓ **Business visualization using Power BI**
- ✓ **End-to-end project execution—from data to deployment**
- ✓ **Documentation and version management using GitHub**

This project closely simulated an industry workflow and helped me build confidence in delivering machine learning solutions that support real business outcomes.



Churn Prediction System — Internship Task 2 Report

By Future Interns

Prepared by: Suyash Shukla

Date: December 2025

1. Introduction

- **Objective:** Predict customers likely to leave a service (churn) using machine learning.
- **Importance:** Retaining customers is more profitable than acquiring new ones, especially in telecom, banking, and SaaS industries.
- **Task Description:**
 - Explore and clean customer dataset
 - Feature engineering
 - Train Logistic Regression, Random Forest, and XGBoost models
 - Evaluate model performance (accuracy, precision, recall, ROC-AUC, PR curves)
 - Visualize insights in Power BI / Matplotlib
 - Generate business-focused insights

```
61 # Step 3.2: Split into X and y
62 X = df_encoded.drop("Churn", axis=1)
63 y = df_encoded["Churn"]
64
65 # Step 3.4: Train-Test Split
66 from sklearn.model_selection import train_test_split
67 X_train, X_test, y_train, y_test = train_test_split(
68     X, y, test_size=0.2, random_state=42
69 )
70
71 print("\ntrain-test split done ✓")
72 print("Training rows:", len(X_train))
73 print("Testing rows:", len(X_test))
74
```

customerID	gender	SeniorCitizen	Partner	...	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0 7590-WAVEG	Female	0	Yes	...	Electronic check	29.85	29.85	0
1 5575-GWDE	Male	0	No	...	Mailed check	56.95	1889.50	0
2 3668-QPYBK	Male	0	No	...	Mailed check	53.85	108.15	1
3 7795-CFOCM	Male	0	No	...	Bank transfer (automatic)	42.30	1849.75	0
4 9237-HQITU	Female	0	No	...	Electronic check	70.70	151.65	1

```
44 # Model Training
45 model = LogisticRegression(max_iter=1000)
46
47 # Train model
48 model.fit(X_train, y_train)
49
50 # Predict on test data
51 y_pred = model.predict(X_test)
52
53 # Accuracy
54 acc = accuracy_score(y_test, y_pred)
55 print(f"Model Accuracy: {acc}")
56
57 # Classification Report
58 print("\nClassification Report:\n")
59 print(classification_report(y_test, y_pred))
60
```

	precision	recall	f1 score	support
0	0.83	0.82	0.80	1883
1	0.22	0.24	0.20	116

2. Software & Libraries Used

- **Software:**
 - Python 3.13 (VS Code IDE)
 - Power BI Desktop
 - Excel (for dataset viewing)
- **Python Libraries:**
 - pandas → Data manipulation & loading
 - numpy → Numerical operations

```
32 df["TotalCharges"] = df["TotalCharges"].replace(" ", pd.NA)
33
34 # Step 2.2: Convert TotalCharges to numeric
35 df["TotalCharges"] = pd.to_numeric(df["TotalCharges"])
36
37 # Step 2.3: Drop rows with missing values
38 df = df.dropna()
39
40 # Step 2.4: Convert Churn column (Yes/No → 1/0)
41 df["churn"] = df["churn"].map({"Yes": 1, "No": 0})
42
43 print("Cleaning Done ✓")
44 print(df.head())
45
```

customerID	gender	SeniorCitizen	Partner	...	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0 7590-WAVEG	Female	0	Yes	...	Electronic check	29.85	29.85	0
1 5575-GWDE	Male	0	No	...	Mailed check	56.95	1889.50	0
2 3668-QPYBK	Male	0	No	...	Mailed check	53.85	108.15	1
3 7795-CFOCM	Male	0	No	...	Bank transfer (automatic)	42.30	1849.75	0
4 9237-HQITU	Female	0	No	...	Electronic check	70.70	151.65	1



- matplotlib → Plotting graphs
- seaborn → Advanced plotting & styling
- scikit-learn → ML models, preprocessing, evaluation metrics
- imbalanced-learn (SMOTE) → Handling class imbalance
- xgboost → Advanced gradient boosting model

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.26100.7171]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin>pip install imbalanced-learn
Collecting imbalanced-learn
  Downloading imbalanced_learn-0.14.0-py3-none-any.whl.metadata (9.8 kB)
Requirement already satisfied: numpy<3, >=1.25.2 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from imbalanced-learn) (2.3.5)
Requirement already satisfied: scipy<2, >=1.11.4 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from imbalanced-learn) (1.16.3)
Requirement already satisfied: scikit-learn<2, >=1.4.2 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from imbalanced-learn) (1.7.2)
Requirement already satisfied: joblib<2, >=1.2.0 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from imbalanced-learn) (1.5.2)
Requirement already satisfied: threadpoolctl<4, >=2.0.0 in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (from imbalanced-learn) (3.6.0)
Downloading imbalanced_learn-0.14.0-py3-none-any.whl (239 kB)
Installing collected packages: imbalanced-learn
Successfully installed imbalanced-learn-0.14.0

[notice] A new release of pip is available: 24.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
```

3. Dataset Overview

- **Dataset Name:** WA_Fn-UseC_-Telco-Customer-Churn.csv
- **Number of Rows:** 7043
- **Number of Columns:** 21
- **Key Columns:** customerID, gender, SeniorCitizen, Contract, PaymentMethod, MonthlyCharges, TotalCharges, Churn
- **Missing Values:** Handled by converting blank TotalCharges to numeric and dropping rows with NaN

4. Data Cleaning & Feature Engineering

- Converted TotalCharges to numeric
- Mapped Churn column: Yes → 1, No → 0
- Dropped customerID column (not useful for ML)
- One-hot encoded all categorical columns (drop_first=True)

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Admin\Desktop\Churn_Prediction_Project> c++; cd 'c:\Users\Admin\
Admin\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\Admin\
n32-x64\bundled\libs\debugpy\launcher' '59932' '--' 'C:\Users\Admin\Desktop\
StreamingMovies 0
Contract 0
PaperlessBilling 0
PaymentMethod 0
MonthlyCharges 0
TotalCharges 0
Churn 0
dtype: int64
PS C:\Users\Admin\Desktop\Churn_Prediction_Project>
```

Core Python Code:

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"].replace(" ", pd.NA))
df = df.dropna()
df["Churn"] = df["Churn"].map({"Yes": 1, "No": 0})
df_encoded = pd.get_dummies(df.drop(columns=["customerID"]), drop_first=True)
```

5. Train-Test Split & Scaling

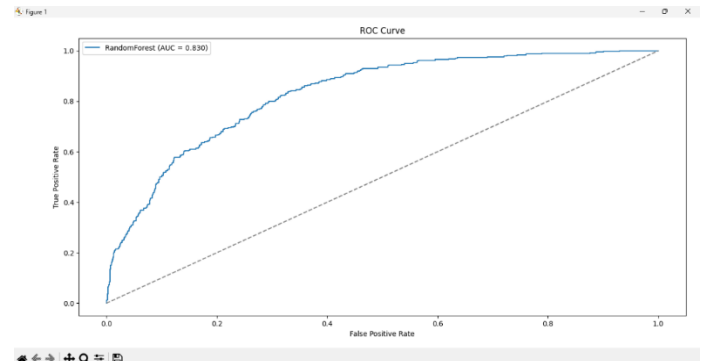
- Split dataset: **80% training, 20% testing**
- StandardScaler applied to scale features
- SMOTE applied to balance classes

Core Python Code:

```
from sklearn.model_selection import  
train_test_split
```

```
from sklearn.preprocessing import  
StandardScaler
```

```
from imblearn.over_sampling import SMOTE
```



```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()
```

```
X_train_scaled =
```

```
scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
smote = SMOTE(random_state=42)
```

```
X_train_bal, y_train_bal =
```

```
smote.fit_resample(X_train_scaled, y_train)
```

```
69 )  
70  
71 print("\nTrain-test split done ✓")  
72 print("Training rows:", len(X_train))  
73 print("Testing rows:", len(X_test))  
74
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Admin\Desktop\Churn_Prediction_Project> cd 'c:\Users\Admin\Desktop\Churn_Prediction_Project'; & 'c:\Users\Admin\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\Admin\.vscode\extensions\ms-python.debugpy-2025.16.0-n32-x64\lib\debugpy\launcher' '56944' '-...' 'C:\Users\Admin\Desktop\Churn_Prediction_Project\churn_analysis.py'
```

Encoding categorical variables...

Encoding Done ✓

	SeniorCitizen	tenure	...	PaymentMethod_Electronic check	PaymentMethod_Mailed check
0	0	1	...	True	False
1	0	34	...	False	True
2	0	2	...	False	True
3	0	45	...	False	False
4	0	2	...	True	False

6. Model Training & Evaluation

6.1 Logistic Regression

- Trained on balanced data
- Accuracy: **~0.7875**
- Classification Report: Precision, Recall, F1-score

```
86  
87 # Train model  
88 model.fit(X_train, y_train)  
89  
90 # Predict on test data  
91 y_pred = model.predict(X_test)  
92  
93 # Accuracy  
94 acc = accuracy_score(y_test, y_pred)  
95 print(f"Model Accuracy: {acc:.4f}")  
96  
97 # Classification report  
98 print("\nClassification Report:\n")  
99 print(classification_report(y_test, y_pred))  
100
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Admin\Desktop\Churn_Prediction_Project> cd 'c:\Users\Admin\Desktop\Churn_Prediction_Project'; & 'c:\Users\Admin\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\Admin\.vscode\extensions\ms-python.debugpy-2025.16.0-n32-x64\lib\debugpy\launcher' '58079' '-...' 'C:\Users\Admin\Desktop\Churn_Prediction_Project\churn_analysis.py'
```

Train-test split done ✓
Training rows: 5625
Testing rows: 1487

Training Logistic Regression Model...

c:\Users\Admin\AppData\Local\Programs\Python\Python313\lib\site-packages\sklearn\linear_model\logistic.py:473: ConvergenceWarning: lbfgs failed to converge after 1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT



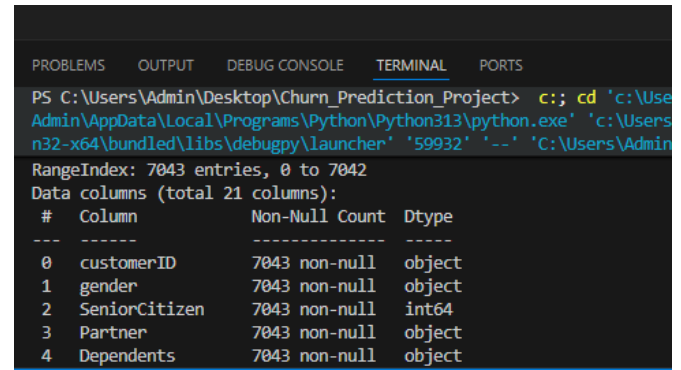
```
from sklearn.linear_model import  
LogisticRegression
```

```
lr_model = LogisticRegression(max_iter=1000)
```

```
lr_model.fit(X_train_bal, y_train_bal)
```

```
y_pred_lr = lr_model.predict(X_test_scaled)
```

- **Confusion Matrix:** Saved as confusion_matrix_lr.png



#	Column	Non-Null Count	Dtype
0	customerID	7043 non-null	object
1	gender	7043 non-null	object
2	SeniorCitizen	7043 non-null	int64
3	Partner	7043 non-null	object
4	Dependents	7043 non-null	object

6.2 Random Forest Classifier

- n_estimators=200
- Accuracy: ~0.75
- Confusion Matrix, Feature Importance saved as PNGs

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf_model = RandomForestClassifier(n_estimators=200, random_state=42)
```

```
rf_model.fit(X_train_bal, y_train_bal)
```

```
y_pred_rf = rf_model.predict(X_test_scaled)
```

```
y_proba_rf = rf_model.predict_proba(X_test_scaled)[: ,1]
```

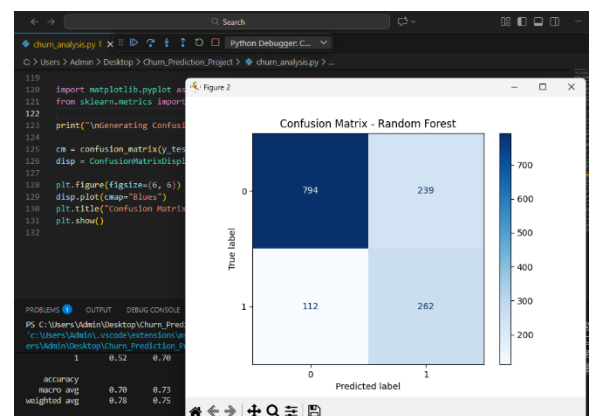
- **ROC Curve:** roc_curve_rf.png
- **Precision-Recall Curve:** precision_recall_rf.png
- **Feature Importance:** Top 20 features → feature_importance_rf.png

6.3 XGBoost Classifier

- n_estimators=200, eval_metric="logloss"
- Accuracy and classification report saved
- ROC comparison chart saved as roc_comparison.png

```
from xgboost import XGBClassifier
```

```
xgb_model = XGBClassifier(n_estimators=200,  
use_label_encoder=False, eval_metric="logloss")
```





```
xgb_model.fit(X_train_bal, y_train_bal)
```

```
y_pred_xgb = xgb_model.predict(X_test_scaled)
```

7. Prediction Results

- Saved **per-customer predictions** with probabilities for Power BI:
- **File:** churn_predictions_for_powerbi.csv
- Columns included:
 - actual_churn
 - pred_lr, prob_lr
 - pred_rf, prob_rf
 - pred_xgb, prob_xgb

Core Python Code:

```
results_df = X_test.copy()
```

```
results_df['actual_churn'] = y_test.reset_index(drop=True)
```

```
results_df['pred_lr'] = y_pred_lr
```

```
results_df['prob_lr'] = lr_model.predict_proba(X_test_scaled)[: ,1]
```

```
results_df.to_csv("churn_predictions_for_powerbi.csv", index=False)
```

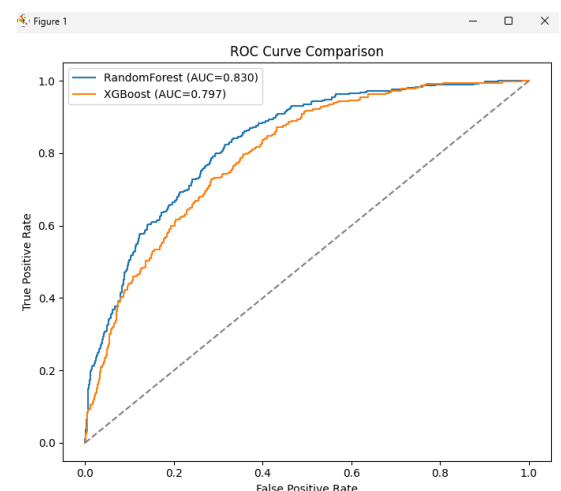
```
Microsoft Windows [Version 10.0.26100.7171]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin>pip install xgboost imbalanced-learn fpdf matplotlib seaborn

Collecting xgboost
  Downloading xgboost-3.1.2-py3-none-win_amd64.whl.metadata (2.1 kB)
Requirement already satisfied: imbalanced-learn in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (0.10.0)
Collecting fpdf
  Downloading fpdf-1.7.2.tar.gz (39 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: matplotlib in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (3.8.0)
Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: numpy in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (1.26.4)
Requirement already satisfied: pandas in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (2.2.2)
Requirement already satisfied: scikit-learn in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (1.5.2)
Requirement already satisfied: joblib in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (1.4.2)
Requirement already satisfied: threadpoolctl in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (3.5.0)
Requirement already satisfied: contourpy in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (1.3.0)
Requirement already satisfied: cycler in c:\users\admin\appdata\local\programs\python\python313\lib\site-packages (0.12.1)
```

8. Power BI Dashboard Insights

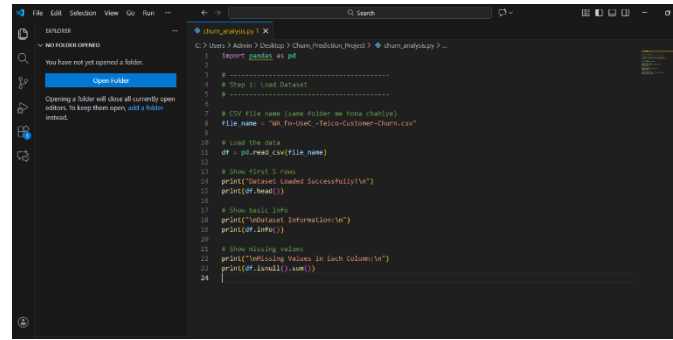
- Overall Churn Rate (Card Visual)
- Churn by Contract Type / Payment Method (Bar Charts)
- Risk Segmentation Table (Top probability customers)
- Imported plots: Feature Importance, Confusion Matrix, ROC & PR Curves



9. Key Observations & Business Recommendations

1. Customers with **Month-to-Month contracts** churn more.

2. **Electronic Check payment method** has higher churn.
3. **Long tenure customers** are less likely to churn.
4. **Target marketing & retention campaigns** for high-risk segments (probability > 0.7).
5. Use **Random Forest / XGBoost model** for predicting churn probability and prioritizing retention strategies.



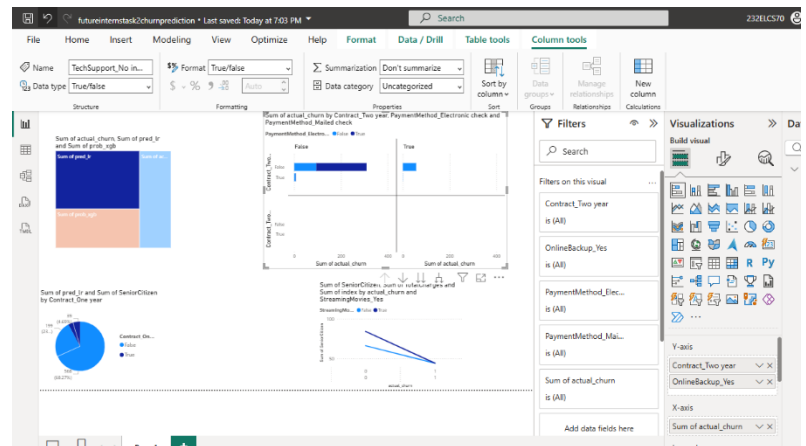
```

1 # Import pandas as pd
2
3 # Step 1: Load Dataset
4
5 # CSV file name (same folder we have change)
6 file_name = "ML for-IsChurn-Teleco-Customer-Churn.csv"
7
8 # Load the data
9 df = pd.read_csv(file_name)
10
11 # Show first 5 rows
12 print("Dataset loaded Successfully!\n")
13 print(df.head())
14
15 # Show basic info
16 print("Dataset Information:\n")
17 print(df.info())
18
19 # Show missing values
20 print("Missing Values in Each column:\n")
21 print(df.isnull().sum())
22
23
24

```

10. Conclusion

- Successfully built a **Churn Prediction System** using Python and Power BI.
- Models trained: Logistic Regression, Random Forest, XGBoost
- All evaluation metrics, visualizations, and feature insights generated
- Ready for business decision-making and retention strategies

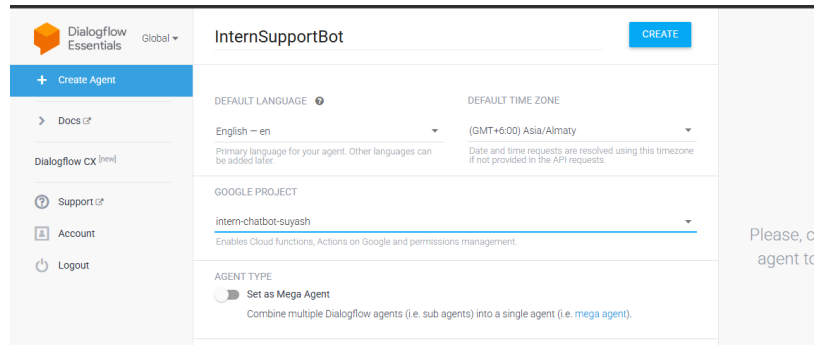


Task: Task 3 – Building a Smart Customer Support Chatbot

1. Introduction

The internship at **Future Interns** provided an opportunity to explore practical applications of **Artificial Intelligence (AI)** and **Natural Language Processing (NLP)** in real-world scenarios.

Task 3 required building a **smart chatbot** capable of handling **customer support queries** in real time. Chatbots are widely used in e-commerce platforms such as **Amazon, Flipkart, and Zomato** to provide instant responses to users, improving customer experience and reducing human intervention.

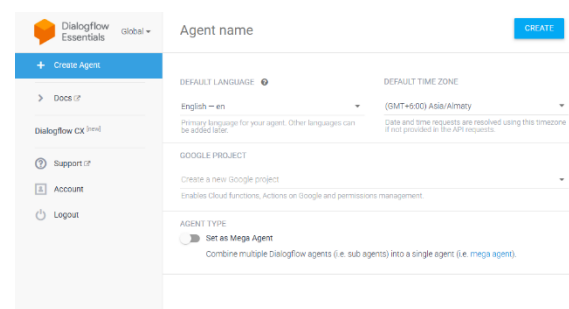
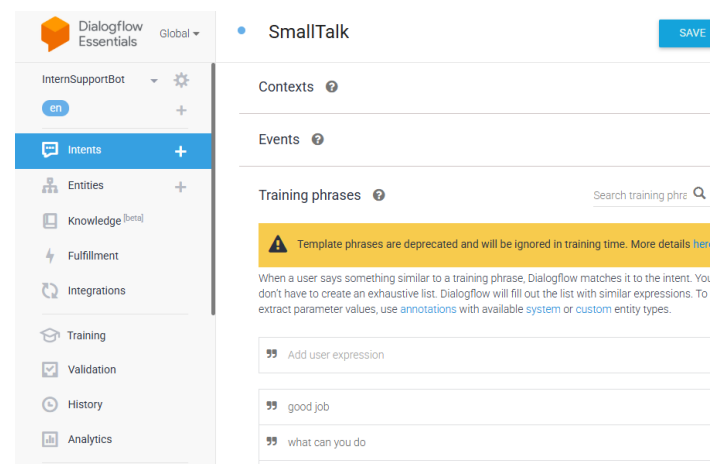


The objective of this task was to gain **hands-on experience** in conversational AI design, entity recognition, intent mapping, and basic chatbot deployment, using modern AI tools like **Dialogflow** and **OpenAI GPT**.

2. Objective

The main objectives of the project were:

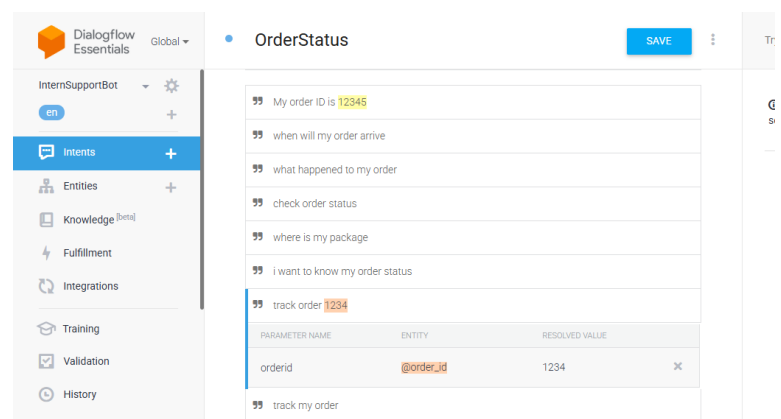
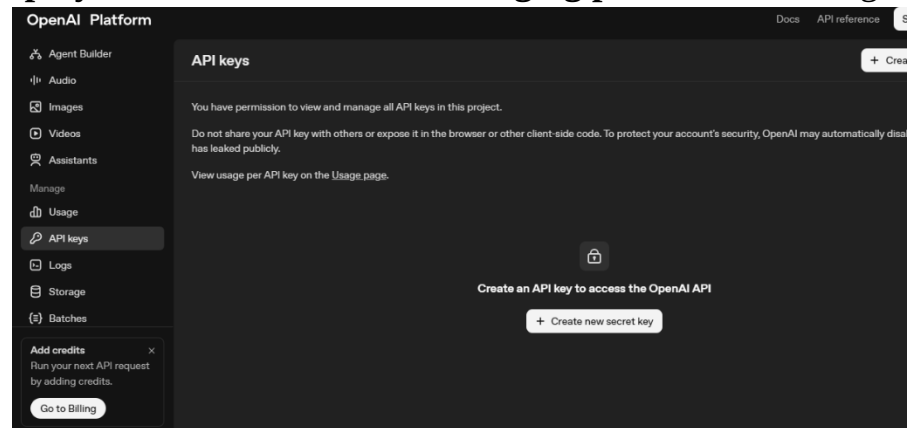
1. To build a **fully functional chatbot** capable of handling customer queries about:
 - Order status
 - Cancellations
 - Refunds
 - Shipping and delivery
 - Returns
2. To **implement small talk and fallback handling**, allowing the bot to respond to greetings, thank you messages, and unrecognized queries.
3. To understand and implement **Dialogflow entities and intents** for real-world chatbot interactions.



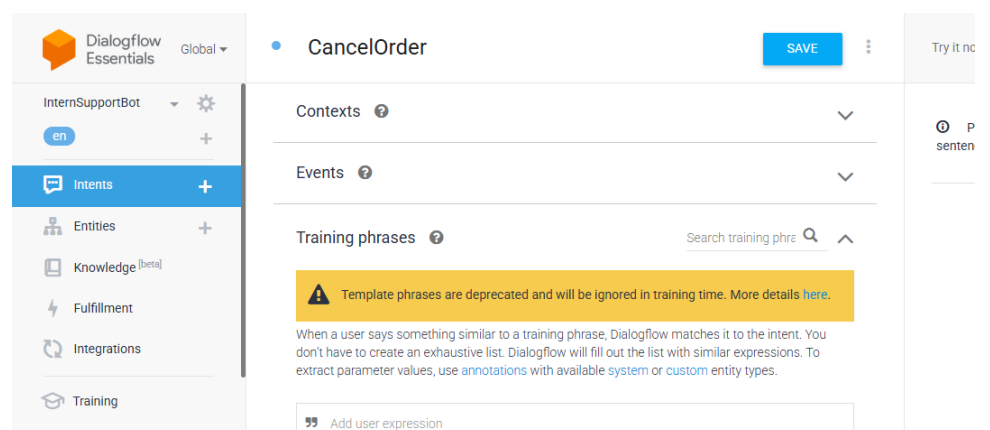
4. To prepare the chatbot for **deployment on websites or messaging platforms**, making it accessible to users without additional coding.

3. Tools and Technologies Used

- **Dialogflow CX / ES** – For designing intents, entities, and conversation flows.
- **Entities** – Custom entity order_id to capture dynamic order numbers.
- **Webhook Integration** – Enabled for transactional intents (OrderStatus, CancelOrder, RefundStatus) for potential backend connection.
- **Web Demo / Embedded Chat** – For testing the chatbot live without coding.
- **Telegram Bot API / Streamlit (Optional)** – For future integrations on messaging platforms or web apps.



- **Sample Dataset:** Customer Support Conversations from Kaggle (used for creating training phrases).



4. Methodology

The project was executed in **step-by-step phases** as outlined below:

4.1 Planning and Dataset Understanding

- Analyzed the **Customer Support Conversations dataset** from Kaggle to identify common queries.
- Categorized queries into **core intents**: order tracking, cancellations, refunds, shipping, returns, and small talk.

- Defined **training phrases** for each intent to enable accurate recognition.

4.2 Creating Intents

Seven main intents were created in Dialogflow, plus one fallback intent:

1. **Welcome Intent** – Handles greetings:

- Training phrases: hi, hello, hey, good morning, hi, I need help
- Response: Hi! 🙌 How can I help you today?

2. **OrderStatus Intent** – Tracks customer orders:

- Training phrases: where is my order, track my order 1234
- Response: Sure! Can you please share your order ID?

3. **CancelOrder Intent** – Handles order cancellations:

- Training phrases: I want to cancel my order, cancel order 1234
- Response: No problem! Please provide your order ID so I can check if cancellation is possible.

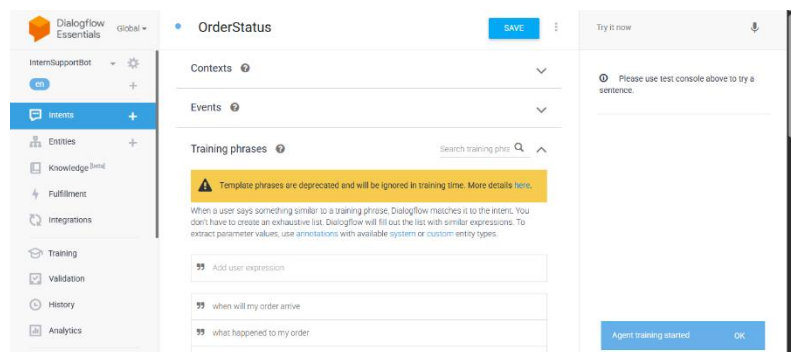
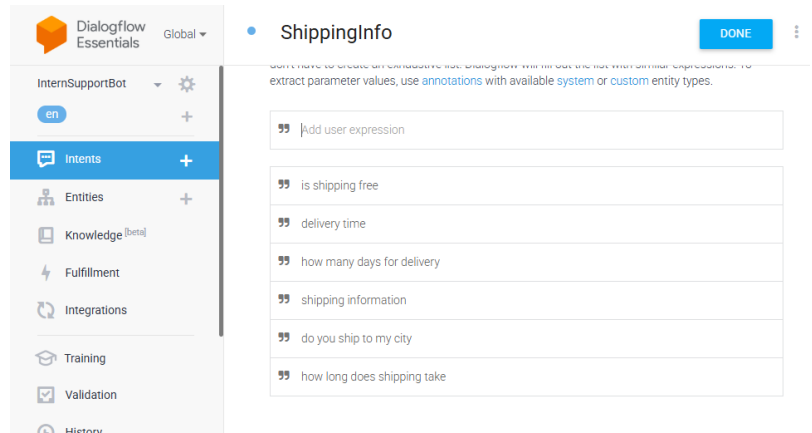
4. **ReturnPolicy Intent** – Handles return queries:

- Training phrases: how can I return an item, return policy
- Response: You can return any product within 7 days of delivery. Would you like help starting a return?

5. **ShippingInfo Intent** – Handles delivery and shipping information:

- Training phrases: how long does shipping take, do you ship to my city
- Response: Our standard delivery time is 3–5 business days. Shipping is free for prepaid orders!

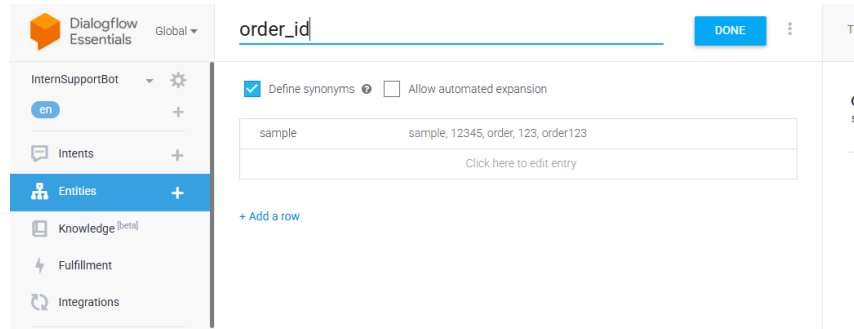
6. **RefundStatus Intent** – Handles refund queries:



- Training phrases: where is my refund, refund status
- Response: Refunds usually take 5–7 business days to reflect. Would you like me to check yours?

7. **SmallTalk Intent** – Handles greetings, thanks, and casual conversations:

- Training phrases: how are you, thanks, good job
- Response: I'm here to help you with anything you need! 😊



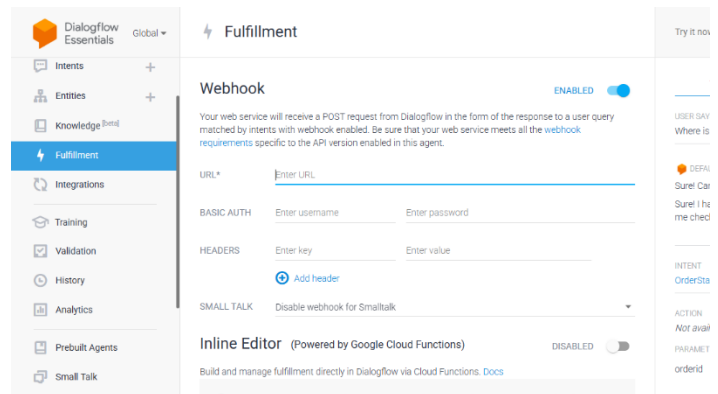
8. **Default Fallback Intent** – Handles unrecognized queries:

- Response: I'm not sure I understood that. Could you please rephrase? 😊

All intents were **tested in the Dialogflow simulator** to ensure correct responses and flow.

4.3 Creating Entities

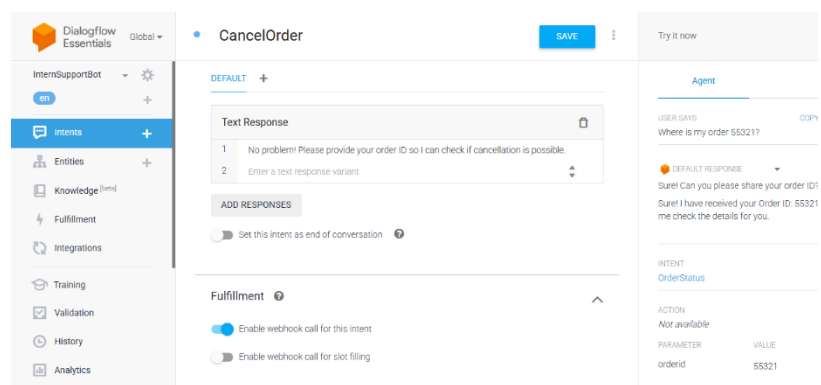
- Created a custom entity **order_id** to capture dynamic order IDs provided by the user.
- Sample value: sample
- Synonyms: 12345, order123
- This entity was linked to **OrderStatus**, **CancelOrder**, and **RefundStatus** intents.



4.4 Webhook Integration

- Webhooks were **enabled for main transactional intents** to prepare for **backend integration**, which allows:

- Dynamic responses based on actual order status
- Handling cancellations and refunds programmatically
- Using OpenAI GPT for smart fallback responses in the future





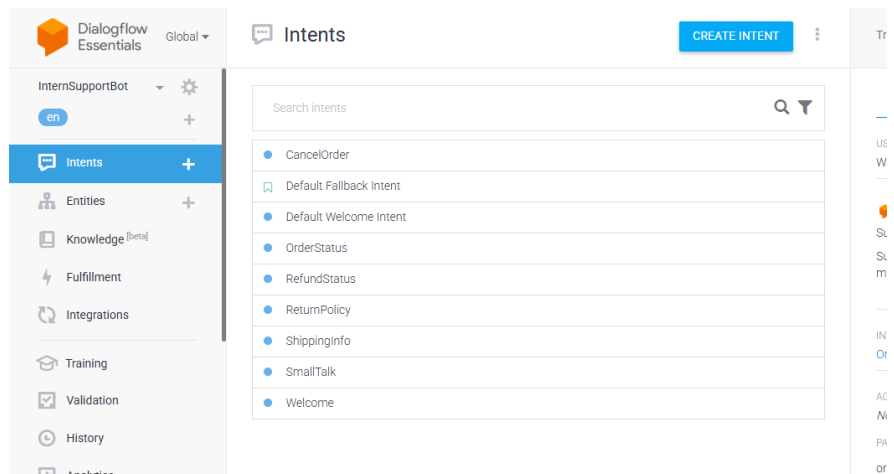
For this project, we used the **Dialogflow Web Demo** for live testing, so no backend coding was required.

4.5 Testing and Validation

- All intents were tested in **Dialogflow simulator**:
 - Checked **order ID recognition** for dynamic input.
 - Verified **responses** for cancellations, refunds, and shipping.
 - Tested **fallback responses** for unrecognized queries.
 - SmallTalk responses verified for casual greetings.
- **Web Demo Integration** allowed live testing via a simple **browser link**:
 - [InternSupportBot Chat Demo](#)

4.6 Deployment (Optional)

- The chatbot can be **embedded on any website** using the provided **iframe code**.
- Can also be **connected to Telegram Bot** or **Streamlit web app** for real-time interactions.



- This makes the bot accessible for **end-users without coding knowledge**.

5. Key Learnings

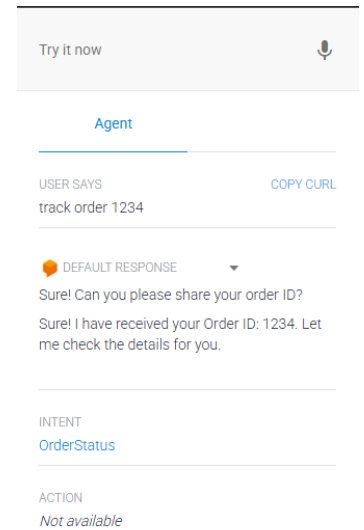
1. **Conversational Design:** Understanding how users interact with bots and designing **intents & responses** accordingly.
2. **Entity Recognition:** Capturing dynamic user inputs like order IDs for transactional intents.
3. **Dialogflow Fulfillment:** Learning how webhooks can extend bot capabilities.
4. **Natural Language Processing:** Training phrases, intent matching, and fallback handling.
5. **Real-time Testing:** Using Web Demo and simulator for validating chatbot functionality.

6. Project Management: Organizing intents, entities, and responses for **real-world chatbot deployment**

6. Challenges Faced

1. **Dynamic Input Handling:** Ensuring the bot could recognize different formats of order IDs.
2. **Fallback Handling:** Designing effective fallback responses that guide users instead of leaving them frustrated.
3. **Testing All Scenarios:** Ensuring that every intent, including small talk, responded correctly.

These challenges were overcome using **training phrases, entity mapping, and Dialogflow simulator testing.**



7. Future Scope

- Connect the chatbot to a **real database or backend API** to fetch actual order details, process cancellations, and refunds.
- Integrate **OpenAI GPT** for **smart fallback responses**, making the bot more human-like.
- Deploy on **Telegram, WhatsApp, or a company website** for real-world usage.
- Add **multilingual support** for wider reach.

Diagnostic info

Raw API response

```
1 {
2   "responseId": "427aa2ac-d0f8-4a0a-8035-a23e750dcd66-bf724aac",
3   "queryResult": {
4     "queryText": "hi",
5     "parameters": {},
6     "allRequiredParamsPresent": true,
7     "fulfillmentText": "Hi! 🙋 How can I help you today?",
8     "fulfillmentMessages": [
9       {
10        "text": {
11          "text": [
12            "Hi! 🙋 How can I help you today?"
13          ]
14        }
15      }
16    ]
17  }
```

CLOSE

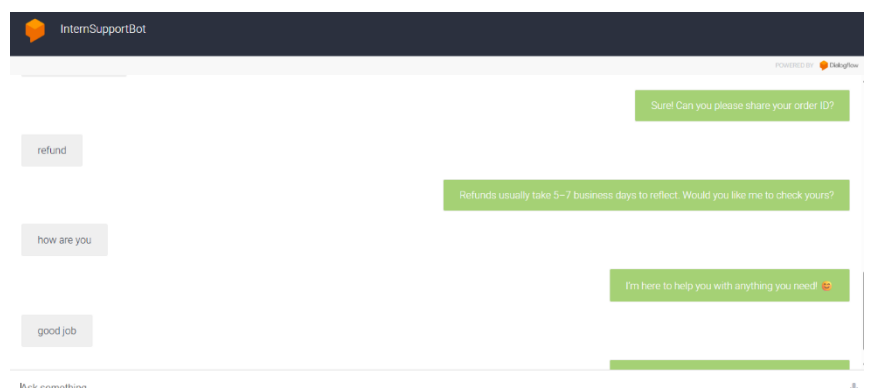
COPY RAW RESPONSE

8. Conclusion

This internship task provided **hands-on experience in AI-powered chatbot development.**

Key takeaways:

- Learned practical **Dialogflow development**, including intents, entities, and webhook configuration.
- Gained insight into **customer support automation** and conversational AI design.





- Prepared a **fully functional, live chatbot** ready for deployment and submission.

Overall, this project strengthened my **NLP, machine learning, and practical AI implementation skills**, making me confident in building real-world **customer support solutions**.

A big thank you to **Future Interns** for providing this amazing opportunity.

9. References

1. Dialogflow Documentation – <https://cloud.google.com/dialogflow/docs>
2. Kaggle Customer Support Conversations Dataset – <https://www.kaggle.com/datasets>
3. YouTube Tutorial: Build a Customer Support Chatbot in Dialogflow.